

[★ Upgrade](#)

Today 6:27 PM

I am a gamer working on a Pygame Tug of War game. I want to fix the input-lock bug where the game stops responding when I rapidly alternate between A and D. Please look at my GitHub repository and the existing code, especially the input handling in `game_engine.py`. Find the reason for this bug and suggest a simple fix so that I can rapidly alternate A and D without the player getting permanently locked. Do not change unrelated parts of the game.

GitHub repository: [shriyacrao/56_tug_of_war](#)

Worked for 11s >

I can inspect the repository, but GitHub access isn't available in this chat right now, so I can't honestly claim I've read your `game_engine.py`.

Please **upload** `game_engine.py` **here** (or paste the input-handling section). Then I can trace the exact A/D bug and give you the **smallest possible change**, without touching unrelated game logic.



```
import random
import pygame
from game.rope import Rope
from game.player import Puller
```

```
class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.rope = Rope(width, height)
```

Show more ▾

Yes — the bug is directly in this input-lock logic. The good news is you only need a **small change in** `handle_event()` .

Why it gets locked

Right now, when you press `A` :

</> Python



▶ Run

```
self.last_key = event.key
self.is_pull_locked = True
```

Then suppose you rapidly do:

A down → A up → D down

The `A KEYUP` should unlock it:

</> Python



▶ Run

```
if event.key == self.last_key:
    self.is_pull_locked = False
```

But with rapid alternating input, the event ordering can leave `is_pull_locked` stuck as `True` because the lock is being used together with `last_key` unnecessarily.

Also, this condition:

</> Python



Run

```
if event.key != self.last_key:
```

means the game is deliberately tracking the previous key rather than simply responding to every valid alternating press.

Simple fix

You don't actually need `is_pull_locked` at all. Since the requirement is "**alternate A and D rapidly**", simply process a pull whenever the player presses a different key from the previous one.

Replace only your `handle_event()` with:

</> Python



Run

```
def handle_event(self, event):
    if self.game_state != "PLAYING":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        return

    if event.type == pygame.KEYDOWN:
        if event.key in (pygame.K_a, pygame.K_d):
            if event.key != self.last_key:
                self.rope.pull_left(1.0)
                self.last_key = event.key
```

And you can remove these two lines from `__init__`:

</> Python



Run

```
self.is_pull_locked = False
```

and from `reset()`:

</> Python



Run

```
self.is_pull_locked = False
```

You can also remove the entire `KEYUP` section because it is no longer needed.

One important thing I noticed

Your current code does this regardless of whether the player presses **A** or **D**:

</> Python



▶ Run

```
self.rope.pull_left(1.0)
```

So both A and D currently pull the rope left.

If that's intentional for your game mechanic—where alternating either key represents the player's pulling action—leave it exactly as it is.

So the minimal fix is essentially:

</> Python



▶ Run

```
if event.type == pygame.KEYDOWN:
    if event.key in (pygame.K_a, pygame.K_d):
        if event.key != self.last_key:
            self.rope.pull_left(1.0)
            self.last_key = event.key
```

This preserves the rest of your game and removes the fragile `KEYUP`-based lock that can cause the permanent input lock.



I am a gamer working on a Pygame Tug of War game. I want to make the computer opponent become more aggressive when the player is getting close to winning. Please check the existing `game_engine.py` and implement a simple dynamic difficulty or "panic surge" mode. When the center flag gets closer to the player's left goal line (`self.rope.left_win_x`), the computer should respond more aggressively by reducing its pull cooldown or increasing its pulling strength. Keep the existing random variation and do not change unrelated parts of the game.

Yes. We can do this without changing the actual tug-of-war mechanics. The idea is to make **visual tension** a **separate value** from the rope's actual position.

What we'll add

- **Rope:** when pulled, tension increases.
- At high tension, the rope gets a small **wave/vibration/sag**.
- Tension gradually fades visually between pulls.
- **Pullers:** briefly lean toward the rope when they pull.
- Computer and player use the same simple animation.
- `pull_left()` / `pull_right()` still move the marker by exactly the same amount as before.

You only need small changes in the three files.

1. `rope.py`

Replace your `Rope` class with this version:

</> Python



▶ Run

```
import math
import pygame

class Rope:
    def __init__(self, screen_width, screen_height):
        self.screen_width = screen_width
        self.screen_height = screen_height
        self.center_y = screen_height // 2
        self.marker_x = screen_width // 2

        self.left_win_x = 180
        self.right_win_x = screen_width - 180
        self.pull_step = 12

        # Visual-only tension
        self.tension = 0.0
        self.tension_time = 0.0

    def pull_left(self, strength=1.0):
        self.marker_x -= int(self.pull_step * strength)

        # Increase visual tension
        self.tension = min(1.0, self.tension + 0.25)
        self.tension_time += 0.8

    def pull_right(self, strength=1.0):
        self.marker_x += int(self.pull_step * strength)

        # Increase visual tension
        self.tension = min(1.0, self.tension + 0.25)
        self.tension_time += 0.8

    def check_winner(self):
        if self.marker_x <= self.left_win_x:
            return "PLAYER"
        if self.marker_x >= self.right_win_x:
            return "COMPUTER"
        return None

    def reset(self):
        self.marker_x = float(self.screen_width // 2)
        self.velocity = 0.0

        # Reset visual tension
        self.tension = 0.0
        self.tension_time = 0.0

    def update_visuals(self):
```

```
# Slowly relax the rope visually.
self.tension = max(0.0, self.tension - 0.025)
self.tension_time += 0.15

def render(self, surface):
    # Base rope coordinates
    start_x = 60
    end_x = self.screen_width - 60

    # Draw the rope as several connected segments.
    # High tension produces a small wave/vibration.
    points = []

    segments = 30

    for i in range(segments + 1):
        t = i / segments
        x = start_x + (end_x - start_x) * t

        # Stronger tension = larger visual movement
        wave_amplitude = 7 * self.tension

        # Small sag in the middle + vibration
        sag = 5 * self.tension * math.sin(math.pi * t)

        wave = (
            wave_amplitude
            * math.sin(self.tension_time * 8 + t * math.pi * 5)
            * math.sin(math.pi * t)
        )

        y = self.center_y + sag + wave
        points.append((int(x), int(y)))

    if len(points) > 1:
        pygame.draw.lines(
            surface,
            (180, 140, 90),
            False,
            points,
            10
        )

    # Green player goal
    pygame.draw.line(
        surface,
        (50, 200, 50),
        (self.left_win_x, self.center_y - 40),
        (self.left_win_x, self.center_y + 40),
        4
    )
```

```

# Red computer goal
pygame.draw.line(
    surface,
    (200, 50, 50),
    (self.right_win_x, self.center_y - 40),
    (self.right_win_x, self.center_y + 40),
    4
)

# Center marker
pygame.draw.line(
    surface,
    (120, 120, 120),
    (self.screen_width // 2, self.center_y - 20),
    (self.screen_width // 2, self.center_y + 20),
    2
)

flag_rect = pygame.Rect(
    int(self.marker_x) - 12,
    self.center_y - 24,
    24,
    48
)

pygame.draw.rect(
    surface,
    (230, 40, 40),
    flag_rect,
    border_radius=4
)

pygame.draw.rect(
    surface,
    (255, 255, 255),
    flag_rect,
    width=2,
    border_radius=4
)

```

Important

The actual mechanics are still:

</> Python



▶ Run

```
self.marker_x -= int(self.pull_step * strength)
```

and

</> Python



▶ Run

```
self.marker_x += int(self.pull_step * strength)
```

So the rope doesn't become harder/easier to pull. The tension is **visual only**.

2. player.py

We can make the puller lean slightly toward the rope.

Replace `render()` with this version:

</> Python



▶ Run

```
def render(self, surface, pulling=False, direction=0):
    """Draw avatar and simple pulling animation."""

    # Lean toward the direction of the rope when pulling.
    lean = 0

    if pulling:
        lean = direction * 8

    body_x = self.x + lean
    body_y = self.y

    # Body
    body_rect = pygame.Rect(
        body_x - 20,
        body_y - 35,
        40,
        70
    )

    pygame.draw.rect(
        surface,
        self.color,
        body_rect,
        border_radius=6
    )

    # Head follows the Lean
    pygame.draw.circle(
        surface,
        (240, 210, 180),
        (body_x, body_y - 50),
        16
    )
```

```

# Name / control tag
label_surf = self.font.render(
    self.label,
    True,
    (240, 240, 240)
)

surface.blit(
    label_surf,
    (
        self.x - label_surf.get_width() // 2,
        self.y + 45
    )
)

```

The direction is:

```

-1 = lean left
+1 = lean right

```



3. game_engine.py

We need to tell the pullers when they are actively pulling.

In `__init__`

Add:

`</>` Python



`▶` Run

```

self.player_pulling = False
self.computer_pulling = False

```

So that part becomes:

`</>` Python



`▶` Run

```

self.last_key = None
self.is_pull_locked = False

self.player_pulling = False
self.computer_pulling = False

self.winner = None
self.game_state = "PLAYING"

```

Modify `handle_event()`

Using your current input system:

</> Python



▶ Run

```
def handle_event(self, event):
    if self.game_state != "PLAYING":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        return

    if event.type == pygame.KEYDOWN:
        if event.key in (pygame.K_a, pygame.K_d):
            if not self.is_pull_locked:
                if event.key != self.last_key:
                    self.rope.pull_left(1.0)

                    self.last_key = event.key
                    self.is_pull_locked = True
                    self.player_pulling = True

    elif event.type == pygame.KEYUP:
        if event.key == self.last_key:
            self.is_pull_locked = False
            self.player_pulling = False
```

This doesn't alter how much the player pulls.

Modify `update()`

Keep your panic-mode code and add the visual updates:

</> Python



▶ Run

```
def update(self):
    if self.game_state != "PLAYING":
        return

    # Update visual rope tension
    self.rope.update_visuals()

    # Pullers stop their active pulling animation
    # unless another key press activates it.
    self.player_pulling = False
    self.computer_pulling = False

    now = pygame.time.get_ticks()
```

```

# Panic surge when the player is close to winning
distance_to_player_goal = self.rope.marker_x - self.rope.left_win_x

if distance_to_player_goal <= 150:
    computer_cooldown = 100
    panic_multiplier = 1.4
else:
    computer_cooldown = self.computer_pull_cooldown
    panic_multiplier = 1.0

if now - self.last_computer_pull >= computer_cooldown:
    computer_variance = random.uniform(0.7, 1.2)
    computer_variance *= panic_multiplier

    self.rope.pull_right(computer_variance)

    self.last_computer_pull = now
    self.computer_pulling = True

result = self.rope.check_winner()

if result:
    self.winner = result
    self.game_state = "GAME_OVER"

```

4. Modify render() in game_engine.py

Change these:

 Python



 Run

```

self.player.render(screen)
self.computer.render(screen)

```

to:

 Python



 Run

```

self.player.render(
    screen,
    pulling=self.player_pulling,
    direction=-1
)

self.computer.render(
    screen,
    pulling=self.computer_pulling,

```

```
direction=1
)
```

The player leans **left**, toward their side of the rope.

The computer leans **right**, toward their side.

One small improvement I'd make

Because `update()` runs every frame, this:

</> Python



Run

```
self.player_pulling = False
self.computer_pulling = False
```

means the leaning animation only appears for roughly one frame when the pull occurs. The rope animation will look good, but the characters may barely visibly lean.

A cleaner visual-only solution is to give the pullers a short animation timer, for example **100–150 ms**, rather than a boolean. That would make the character visibly jerk/lean with every pull while still keeping the game mechanics completely unchanged.

If you want the effect to feel more like an actual **"STRUGGLE!" tug-of-war animation**, I'd use that timer approach rather than making the characters stay leaned while a key is held.



I want the Task 3 animation to be simple and visible without bringing back the input-lock bug we already fixed in Task 1.

Please revise your previous solution. Keep the Task 1 input handling where A/D pulls whenever the key is different from `last_key`, without `is_pull_locked` or the `KEYUP` locking logic. For the puller animation, use a short visual timer so the player and computer visibly lean for a brief moment after each pull. Keep the rope tension animation visual-only and do not change the actual pulling mechanics. Give me only the changes needed in `rope.py`, `player.py`, and `game_engine.py`.

👍 Yes — this version keeps the **Task 1 input fix exactly as it is**, and Task 3 only adds visual effects. No `is_pull_locked`, no KEYUP locking, and no changes to how far the rope moves.

`rope.py`

Replace your `rope.py` with:

</> Python



▶ Run

```
import math
import pygame

class Rope:
    def __init__(self, screen_width, screen_height):
        self.screen_width = screen_width
        self.screen_height = screen_height
        self.center_y = screen_height // 2
        self.marker_x = screen_width // 2

        self.left_win_x = 180
        self.right_win_x = screen_width - 180
        self.pull_step = 12

        # Visual-only tension
        self.tension = 0.0
        self.tension_time = 0.0

    def pull_left(self, strength=1.0):
        # Actual pulling mechanic stays unchanged
        self.marker_x -= int(self.pull_step * strength)

        # Visual tension only
        self.tension = min(1.0, self.tension + 0.25)
        self.tension_time += 0.8

    def pull_right(self, strength=1.0):
        # Actual pulling mechanic stays unchanged
        self.marker_x += int(self.pull_step * strength)

        # Visual tension only
        self.tension = min(1.0, self.tension + 0.25)
        self.tension_time += 0.8

    def check_winner(self):
        if self.marker_x <= self.left_win_x:
            return "PLAYER"
```

```
if self.marker_x >= self.right_win_x:
    return "COMPUTER"

return None

def reset(self):
    self.marker_x = float(self.screen_width // 2)
    self.tension = 0.0
    self.tension_time = 0.0

def update_visuals(self):
    # Slowly reduce visual tension between pulls
    self.tension = max(0.0, self.tension - 0.025)
    self.tension_time += 0.15

def render(self, surface):
    start_x = 60
    end_x = self.screen_width - 60

    # Draw rope as connected segments so it can visibly move
    points = []
    segments = 30

    for i in range(segments + 1):
        t = i / segments

        x = start_x + (end_x - start_x) * t

        # Visual-only wave and sag
        wave_amplitude = 7 * self.tension
        sag = 5 * self.tension * math.sin(math.pi * t)

        wave = (
            wave_amplitude
            * math.sin(self.tension_time * 8 + t * math.pi * 5)
            * math.sin(math.pi * t)
        )

        y = self.center_y + sag + wave
        points.append((int(x), int(y)))

    if len(points) > 1:
        pygame.draw.lines(
            surface,
            (180, 140, 90),
            False,
            points,
            10
        )

    # Player win line
    pygame.draw.line(
```

```
        surface,
        (50, 200, 50),
        (self.left_win_x, self.center_y - 40),
        (self.left_win_x, self.center_y + 40),
        4
    )

    # Computer win line
    pygame.draw.line(
        surface,
        (200, 50, 50),
        (self.right_win_x, self.center_y - 40),
        (self.right_win_x, self.center_y + 40),
        4
    )

    # Center line
    pygame.draw.line(
        surface,
        (120, 120, 120),
        (self.screen_width // 2, self.center_y - 20),
        (self.screen_width // 2, self.center_y + 20),
        2
    )

    # Marker / flag
    flag_rect = pygame.Rect(
        int(self.marker_x) - 12,
        self.center_y - 24,
        24,
        48
    )

    pygame.draw.rect(
        surface,
        (230, 40, 40),
        flag_rect,
        border_radius=4
    )

    pygame.draw.rect(
        surface,
        (255, 255, 255),
        flag_rect,
        width=2,
        border_radius=4
    )
```

Only change `render()` so it can visually lean during a pull.

</> Python



▶ Run

```
import pygame

class Puller:
    """Represents a puller character anchor on either side of the rope."""

    def __init__(self, x, y, color, label):
        self.x = x
        self.y = y
        self.color = color
        self.label = label
        self.font = pygame.font.SysFont(None, 24)

    def render(self, surface, pulling=False, direction=0):
        """Draw avatar and label."""

        # Small visual lean during a pull
        lean = direction * 8 if pulling else 0

        body_x = self.x + lean
        body_y = self.y

        # Body
        body_rect = pygame.Rect(
            body_x - 20,
            body_y - 35,
            40,
            70
        )

        pygame.draw.rect(
            surface,
            self.color,
            body_rect,
            border_radius=6
        )

        # Head
        pygame.draw.circle(
            surface,
            (240, 210, 180),
            (body_x, body_y - 50),
            16
        )

        # Name / control tag
        label_surf = self.font.render(
```

```

        self.label,
        True,
        (240, 240, 240)
    )

    surface.blit(
        label_surf,
        (
            self.x - label_surf.get_width() // 2,
            self.y + 45
        )
    )

```

game_engine.py

1. In `__init__`

Keep your existing Task 1 code, including:

`</> Python`



`▶ Run`

```
self.last_key = None
```

Do not add `is_pull_locked`.

Add these two timers after your existing computer-pull variables:

`</> Python`



`▶ Run`

```
self.player_pull_until = 0
self.computer_pull_until = 0
```

So that section becomes:

`</> Python`



`▶ Run`

```

self.last_key = None
self.winner = None
self.game_state = "PLAYING"

self.computer_pull_cooldown = 180
self.last_computer_pull = pygame.time.get_ticks()

# Short visual animation timers
self.player_pull_until = 0
self.computer_pull_until = 0

```

2. Replace `handle_event()`

This keeps the **Task 1 input handling** exactly as requested:

</> Python



▶ Run

```
def handle_event(self, event):
    if self.game_state != "PLAYING":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        return

    if event.type == pygame.KEYDOWN:
        if event.key in (pygame.K_a, pygame.K_d):
            if event.key != self.last_key:
                self.rope.pull_left(1.0)
                self.last_key = event.key

            # Start short player pull animation
            self.player_pull_until = pygame.time.get_ticks() + 140
```

There is **no** `KEYUP` section and **no** `is_pull_locked`.

So rapidly alternating:

A → D → A → D → A → D



continues to work.

3. Update `update()`

Keep your existing computer difficulty/panic logic. Add the visual updates shown below:

</> Python



▶ Run

```
def update(self):
    if self.game_state != "PLAYING":
        return

    now = pygame.time.get_ticks()

    # Update rope visuals only
    self.rope.update_visuals()

    distance_to_player_goal = self.rope.marker_x - self.rope.left_win_x

    if distance_to_player_goal <= 150:
        computer_cooldown = 100
```

```

        panic_multiplier = 1.4
    else:
        computer_cooldown = self.computer_pull_cooldown
        panic_multiplier = 1.0

    if now - self.last_computer_pull >= computer_cooldown:
        computer_variance = random.uniform(0.7, 1.2)
        computer_variance *= panic_multiplier

        self.rope.pull_right(computer_variance)
        self.last_computer_pull = now

        # Start short computer pull animation
        self.computer_pull_until = now + 140

    result = self.rope.check_winner()

    if result:
        self.winner = result
        self.game_state = "GAME_OVER"

```

The important part is that `tension` **does not affect** `marker_x`. It only affects how the rope is drawn.

4. Update `reset()`

Remove any `is_pull_locked` reset.

Use:

</> Python



▶ Run

```

def reset(self):
    self.rope.reset()
    self.last_key = None
    self.player_pull_until = 0
    self.computer_pull_until = 0
    self.winner = None
    self.game_state = "PLAYING"
    self.last_computer_pull = pygame.time.get_ticks()

```

5. Change only the puller rendering lines

In `render()`, replace:

</> Python



▶ Run

```
self.player.render(screen)
self.computer.render(screen)
```

with:

</> Python



▶ Run

```
now = pygame.time.get_ticks()

player_pulling = now < self.player_pull_until
computer_pulling = now < self.computer_pull_until

self.player.render(
    screen,
    pulling=player_pulling,
    direction=-1
)

self.computer.render(
    screen,
    pulling=computer_pulling,
    direction=1
)
```

Everything else in your `render()` stays the same.

Result

- **Task 1:** A/D alternating still works correctly.
- **Player:** briefly leans toward the rope after every accepted A/D pull.
- **Computer:** briefly leans toward the rope after each computer pull.
- **Rope:** briefly waves/sags when tension increases, then settles.
- **Actual mechanics:** `marker_x` , pull strength, win conditions, and computer behavior remain unchanged.
- **No input lock:** no `is_pull_locked` and no `KEYUP` logic.



I am a gamer working on a Pygame Tug of War game. I want to add a 45-second match timer and a Sudden Death mode.

Please check the existing `game_engine.py` and add a visible timer at the top of the game. The match should

start with 45 seconds. If neither player wins when the 45 seconds ends, display "Sudden Death" and make both the player's keystroke pulls and the computer's pulls move the rope twice as far as normal. Also make sure pressing R resets the timer and exits Sudden Death. Keep the implementation simple and do not change unrelated game parts.

Yes. Keep the existing Task 1 input handling and Task 3 animation exactly as they are. Add only the timer/Sudden Death logic below.

game_engine.py

1. Add these variables in `__init__`

After your existing game-state variables:

</> Python



▶ Run

```
self.winner = None
self.game_state = "PLAYING"

# Match timer
self.match_duration = 45000 # 45 seconds
self.match_start_time = pygame.time.get_ticks()
self.sudden_death = False
```

So you have:

</> Python



▶ Run

```
self.last_key = None
self.winner = None
self.game_state = "PLAYING"

# Match timer
self.match_duration = 45000
self.match_start_time = pygame.time.get_ticks()
self.sudden_death = False
```

2. Update `handle_event()`

Keep your existing Task 1 A/D logic, but change the actual player pull line to account for Sudden Death:

</> Python



▶ Run

```
def handle_event(self, event):
    if self.game_state != "PLAYING":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        return

    if event.type == pygame.KEYDOWN:
        if event.key in (pygame.K_a, pygame.K_d):
            if event.key != self.last_key:
                pull_strength = 2.0 if self.sudden_death else 1.0

                self.rope.pull_left(pull_strength)
                self.last_key = event.key

            # Start short player pull animation
            self.player_pull_until = pygame.time.get_ticks() + 140
```

This preserves:

</> Python



▶ Run

```
if event.key != self.last_key:
```

so the input-lock bug does **not** come back.

3. Update update()

Add the timer at the beginning of your existing update() :

</> Python



▶ Run

```
def update(self):
    if self.game_state != "PLAYING":
        return

    now = pygame.time.get_ticks()

    # Update rope visuals only
    self.rope.update_visuals()

    # Check match timer
    if not self.sudden_death:
        elapsed = now - self.match_start_time

        if elapsed >= self.match_duration:
            self.sudden_death = True
```

```

distance_to_player_goal = self.rope.marker_x - self.rope.left_win_x

if distance_to_player_goal <= 150:
    computer_cooldown = 100
    panic_multiplier = 1.4
else:
    computer_cooldown = self.computer_pull_cooldown
    panic_multiplier = 1.0

if now - self.last_computer_pull >= computer_cooldown:
    computer_variance = random.uniform(0.7, 1.2)
    computer_variance *= panic_multiplier

    # Double computer pull during Sudden Death
    if self.sudden_death:
        computer_variance *= 2.0

    self.rope.pull_right(computer_variance)
    self.last_computer_pull = now

    # Start short computer pull animation
    self.computer_pull_until = now + 140

result = self.rope.check_winner()

if result:
    self.winner = result
    self.game_state = "GAME_OVER"

```

The important part is:

</> Python



▶ Run

```

if self.sudden_death:
    computer_variance *= 2.0

```

So the normal computer behavior remains unchanged before Sudden Death.

4. Update reset()

Replace your current reset() with:

</> Python



▶ Run

```

def reset(self):
    self.rope.reset()
    self.last_key = None
    self.player_pull_until = 0

```

```

self.computer_pull_until = 0

self.winner = None
self.game_state = "PLAYING"

self.match_start_time = pygame.time.get_ticks()
self.sudden_death = False

self.last_computer_pull = pygame.time.get_ticks()

```

This makes `R` :

- reset the rope
- reset the 45-second timer
- exit Sudden Death
- reset the pull animations

5. Add the visible timer in `render()`

At the beginning of your `render()` , after:

</> Python



▶ Run

```
screen.fill((30, 32, 36))
```

add:

</> Python



▶ Run

```

now = pygame.time.get_ticks()

if self.sudden_death:
    timer_text = "SUDDEN DEATH"
else:
    remaining = max(
        0,
        (self.match_duration - (now - self.match_start_time) + 999) // 1000
    )
    timer_text = f"Time: {remaining}s"

timer_surf = self.font_small.render(
    timer_text,
    True,
    (240, 240, 240)
)

screen.blit(
    timer_surf,

```

```
(  
    self.width // 2 - timer_surf.get_width() // 2,  
    10  
)  
)
```

So the top of the screen will show approximately:

Time: 45s



then:

Time: 32s



and when the timer reaches zero:

SUDDEN DEATH



What changes during Sudden Death

Only the pull strength changes:

Normal:

Player → 1.0

Computer → normal variance

Sudden Death:

Player → 2.0

Computer → normal variance × 2



Your rope animation, puller leaning, A/D alternating behavior, computer panic logic, and win conditions remain otherwise unchanged.



ChatGPT can make mistakes. Check important info.